

A Technique for Drawing Directed Graphs

Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Kiem-Phong Vo

Abstract—We describe a four-pass algorithm for drawing directed graphs. The first pass finds an optimal rank assignment using a network simplex algorithm. The second pass sets the vertex order within ranks by an iterative heuristic incorporating a novel weight function and local transpositions to reduce crossings. The third pass finds optimal coordinates for nodes by constructing and ranking an auxiliary graph. The fourth pass makes splines to draw edges. The algorithm makes good drawings and runs fast.

Index Terms—Directed graph layout algorithm, network simplex.

I. INTRODUCTION

DRAWING abstract graphs is a topic of ongoing research, having such applications as visualization of programs and data structures and document preparation. This paper describes a technique for drawing directed graphs in the plane. The goal is to make high-quality drawings quickly enough for interactive use. These algorithms are the basis of a practical implementation [1].

A. Aesthetic Criteria

To make drawings, it helps to assume that a directed graph has an overall flow or direction, such as top to bottom (assumed in most examples in this paper) or left to right. Such flows can be seen in handmade drawings of finite automata where the flow is from initial to terminal states, or in data flow graphs from input to output. This observation has motivated a collection of methods for drawing digraphs based on the following aesthetic principles:

A1: Expose hierarchical structure in the graph. In particular, aim edges in the same general direction if possible. This aids finding directed paths and highlights source and sink nodes.

A2: Avoid visual anomalies that do not convey information about the underlying graph. For example, avoid edge crossings and sharp bends.

A3: Keep edges short. This makes it easier to find related nodes and contributes to A2.

A4: Favor symmetry and balance. This aesthetic has a secondary role in a few places in our algorithm.

There is no way to optimize all these aesthetics simultaneously. For instance, a placement of nodes and orientation of edges preferred according to A1 may force edge crossings that are undesirable according to A2. What is more, it is computationally intractable to minimize edge crossings or to find subgraphs having symmetry. We therefore make some

```

1. procedure draw_graph()
2. begin
3.   rank();
4.   ordering();
5.   position();
6.   make_splines();
7. end

```

Fig. 1. Main algorithm.

simplifying assumptions and rely on heuristics that run quickly and make good layouts in common cases. For a survey of other aesthetic principles, we refer the reader to the annotated bibliography on graph-drawing algorithms [2].

B. Problem Description

The input to the drawing algorithm is an attributed graph $G = (V, E)$ possibly containing loops and multiedges. We assume that G is connected, as each connected component can be laid out separately. The attributes are:

$xsize(v), ysize(v)$:	Size of bounding box of a node v .
$nodesep(G)$:	Minimum horizontal separation between node boxes.
$ranksep(G)$:	Minimum vertical separation between node boxes.
$\omega(e)$:	Weight of an edge e , usually 1. The weight signifies the edge's importance, which translates to keeping the edge short and vertically aligned.

The algorithm assigns each node v to a rectangle in the plane with the center point $(x(v), y(v))$ and assigns each edge e to a sequence of B -spline control points $(x_0(e), y_0(e)), \dots, (x_n(e), y_n(e))$. Although the unit of these dimensions is not specified, it is convenient to use the traditional coordinate system of 72 units per inch in an implementation. The layout is generally guided by the aesthetic criteria A1–A4, and specifically by the graph attributes. The details of these constraints will be supplied in the following sections.

The user can further constrain the layout in a way that is useful for drawing graphs that have time-lines or for highlighting source and sink nodes. The initial pass of the algorithm described in the next section assigns nodes to discrete ranks $0 \dots Max.rank$. Nodes in the same rank receive the same Y coordinate value. The user may provide sets $S_{max}, S_{min}, S_0, S_1, \dots, S_k \subseteq V$. These are (possibly

Manuscript received August 26, 1991; revised May 22, 1992. Recommended by Helmut Partsch.

The authors are with AT&T Bell Laboratories, Murray Hill, NJ 07974.

IEEE Log Number 9206903.

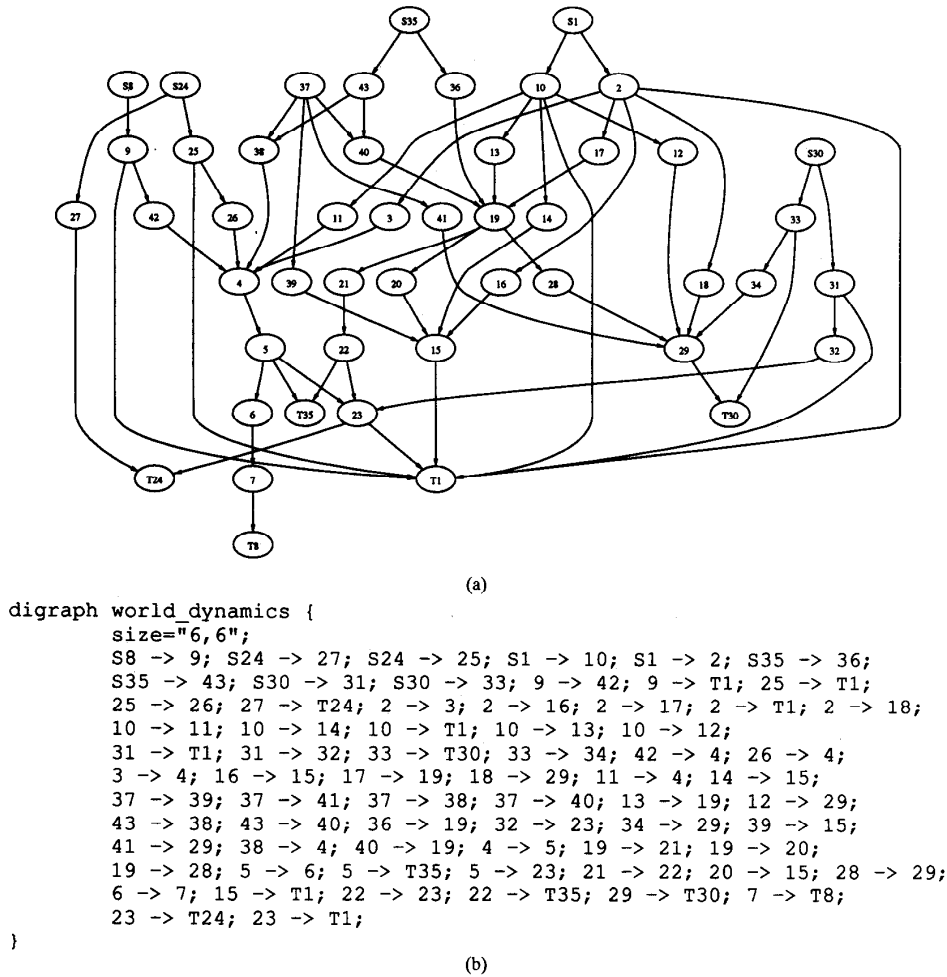


Fig. 2. (a) 1.11 s user time on a Sun-4/280. (b) Graph file listing.

empty) sets of nodes that must be placed together on the maximum, minimum, or same rank, respectively.

C. Related Work

Drawing digraphs using an iterative method to reduce edge crossing was first studied by Warfield [3], and similar methods were discovered by Carpano [4] and Sugiyama, Tagawa, and Toda [5]. Di Battista and Tamassia describe an algorithm for embedding planar acyclic digraphs such that all edges flow in the same direction [6]. We view our work as building on the approach of Warfield *et al.*

D. Overview

The graph-drawing algorithm has four passes, as shown in Fig. 1. The first pass places the nodes in discrete ranks. The second sets the order of nodes within ranks to avoid edge crossings. The third sets the actual layout coordinates of nodes. The final pass finds the spline control points for edges.

Our contributions are 1) an efficient way of ranking the nodes using a network simplex algorithm, 2) improved heuris-

tics to reduce edge crossings, 3) a method for computing the node coordinates as a rank assignment problem, and 4) a method for setting spline control points. Techniques 1) and 2) were first implemented in the graph drawing program *dag*, described in [1]. Further work, especially 3) and 4), have been incorporated in *dot* [7], a successor to *dag*. Figs. 2 and 3 are samples of *dot*'s output with the corresponding input files.

II. OPTIMAL RANK ASSIGNMENT

The first pass assigns each node $v \in G$ to an integer rank $\lambda(v)$ consistent with its edges. This means that for every $e = (v, w) \in E$, $\lambda(w) \geq \lambda(v)$, where the *length* $l(e)$ of $e = (v, w)$ is defined as $\lambda(w) - \lambda(v)$, and $\delta(e)$ represents some given *minimum length* constraint. $\delta(e)$ is usually 1, but can take any nonnegative integer value. $\delta(e)$ may be set internally for technical reasons as described shortly or externally if the user wants to adjust the rank assignment. For this pass, each of the nonempty sets $S_{\max}, S_{\min}, S_0, \dots, S_k$ is temporarily merged

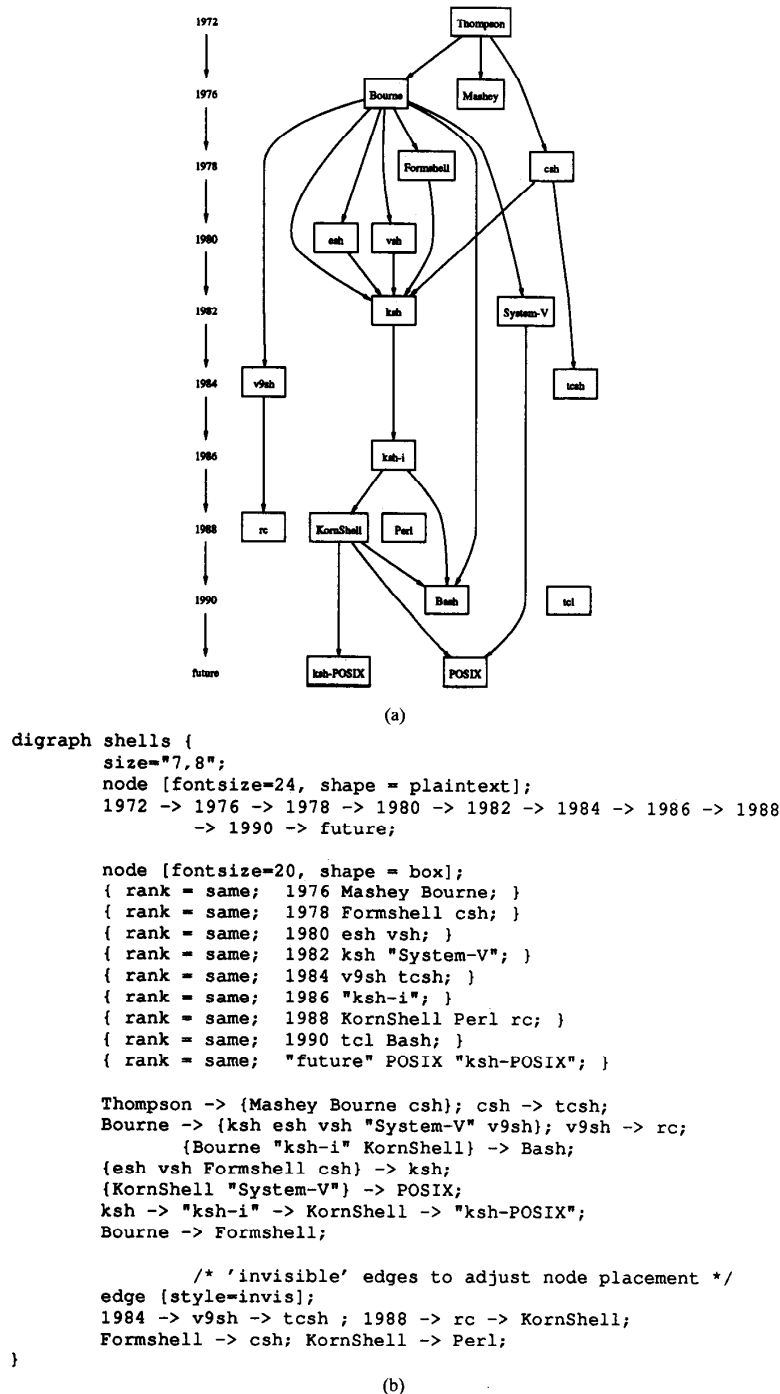


Fig. 3. (a) 0.50 s user time of 0.50 s on a Sun-4/280. (b) Graph file listing.

into one node. In addition, loops are ignored, and multiple edges are merged into one edge whose weight is the sum of the weights of the merged edges. For efficiency, leaf nodes that are not a member of one of the above sets may be ignored, since the rank of a leaf is trivially determined in an optimal ranking.

A. Making the Graph Acyclic

A graph must be acyclic to have a consistent rank assignment. Because the input graph may contain cycles, a preprocessing step detects cycles and breaks them by reversing certain edges [8]. Of course, these edges are only reversed

internally; arrowheads in the drawing show the original direction. A useful procedure for breaking cycles is based on depth-first search. Edges are searched in the “natural order” of the graph input, starting from some source or sink nodes if any exist. Depth-first search partitions edges into two sets: tree edges and nontree edges [9]. The tree defines a partial order on nodes. Given this partial order, the nontree edges further partition into three sets: cross edges, forward edges, and back edges. Cross edges connect unrelated nodes in the partial order. Forward edges connect a node to some of its descendants. Back edges connect a descendant to some of its ancestors. It is clear that adding forward and cross edges to the partial order does not create cycles. Because reversing back edges makes them into forward edges, all cycles are broken by this procedure.

It seems reasonable to try to reverse a smaller or even minimal set of edges. One difficulty is that finding a minimal set (the “feedback arc set” problem) is NP-complete [10], [11]. More important, this would probably not improve the drawings. We implemented a heuristic to reverse edges that participate in many cycles. The heuristic takes one nontrivial strongly connected component at a time, in an arbitrary order. With each component, it counts the number of times each edge forms a cycle in a depth-first traversal. An edge with a maximal count is reversed. This is repeated until there are no more nontrivial strongly connected components.

Experiments with this heuristic show that most directed graphs arising from practical applications have a natural edge direction even when they contain cycles. Graph input usually reflects this natural direction. In fact, graphs are often created by a graph search performed by some other tool. Reversing an inappropriate edge disturbs the drawing. For instance, even when a procedure call graph has cycles, one still expects to see top-level functions near the top of the drawing, and not somewhere in the middle. From the standpoint of stability, the depth-first, cycle-breaking heuristic seems preferable. It also makes more informative drawings than would be obtained by collapsing all the nodes in a cycle into one node, or placing the nodes in a cycle on the same rank, or duplicating one of the nodes in the cycle, as various researchers have suggested [4], [12], [5].

Another detail is that the nodes representing $S_{\max}$ and $S_{\min}$ must always have the maximum and minimum rank assignments. This property is ensured by reversing out-edges of $S_{\max}$ and in-edges of $S_{\min}$. Also, for all nodes v with no in-edge, we make a temporary edge $(S_{\min}, v)$ with $\delta = 0$, and for all nodes v with no out-edge, we make a temporary edge $(v, S_{\max})$ with $\delta = 0$. Thus, $\lambda(S_{\min}) \leq \lambda(v) \leq \lambda(S_{\max})$ for all v .

B. Problem Definition

Principle A3 prescribes making short edges. Besides making better layouts, short edges reduce the running time of later passes whose time depends on the total edge length. So it is desirable to find an optimal node ranking, i.e., one for which the sum of all the weighted edge lengths is minimal.

Finding an optimal ranking can be reformulated as the following integer program:

$$\begin{aligned} \min \quad & \sum_{(v,w) \in E} \omega(v,w)(\lambda(w) - \lambda(v)) \\ \text{subject to: } & \lambda(w) - \lambda(v) \geq \delta(v,w) \forall (v,w) \in E \end{aligned}$$

The *weight* function ω and the *minimum length* function δ as previously described map the edge set E into the nonnegative rational numbers and the nonnegative integers, respectively.

There are various ways to solve this integer program in polynomial time. One method is to solve the equivalent linear program, then transform the solution to an integer one in polynomial time. Another involves converting the optimal rank assignment problem to an equivalent min-cost flow or circulation problem, for which there are polynomial-time algorithms (see [13] and its references). As the constraint matrix is totally unimodular, the problem can also be solved, though not necessarily in polynomial time, by applying the simplex method. A more complete discussion of these and other techniques will be reported in [14].

C. Network Simplex

Here, we describe a simple approach to the problem based on a network simplex formulation [15]. Although its time complexity has not been proven polynomial, in practice it takes few iterations and runs quickly.

We begin with a few definitions and observations. A *feasible* ranking is one satisfying the length constraints $\ell(e) \geq \delta(e)$ for all e . Given any ranking, not necessarily feasible, the *slack* of an edge is the difference of its length and its minimum length. Thus, a ranking is feasible if the slack of every edge is nonnegative. An edge is *tight* if its slack is zero.

A spanning tree of a graph induces a ranking, or rather, a family of equivalent rankings. (Note that the spanning tree is on the underlying unrooted undirected graph, and is not necessarily a directed tree.) This ranking is generated by picking an initial node and assigning it a rank. Then, for each node adjacent in the spanning tree to a ranked node, assign it the rank of the adjacent node, incremented or decremented by the minimum length of the connecting edge, depending on whether it is the head or tail of the connecting edge. This process is continued until all nodes are ranked. A spanning tree is *feasible* if it induces a feasible ranking. By construction, all edges in the feasible tree are tight.

Given a feasible spanning tree, we can associate an integer *cut value* with each tree edge as follows. If the tree edge is deleted, the tree breaks into two connected components, the tail component containing the tail node of the edge, and the head component containing the head node. The cut value is defined as the sum of the weights of all edges from the tail component to the head component, including the tree edge, minus the sum of the weights of all edges from the head component to the tail component.

Typically (but not always because of degeneracy), a negative cut value indicates that the weighted edge length sum could be reduced by lengthening the tree edge as much as possible, until one of the head component to tail component edges

```

1. procedure rank()
2.   feasible_tree();
3.   while (e = leave_edge())  $\neq$  nil do
4.     f = enter_edge(e);
5.     exchange(e,f);
6.   end
7.   normalize();
8.   balance();
9. end

```

Fig. 4. Network simplex.

becomes tight. This corresponds to replacing the tree edge in the spanning tree with the newly tight edge, obtaining a new feasible spanning tree. It is also simple to see that an optimal ranking can be used to generate another optimal ranking induced by a feasible spanning tree. These observations are the key to solving the ranking problem in a graphical rather than algebraic context. Tree edges with negative cut values are replaced by appropriate nontree edges, until all tree edges have nonnegative cut values. To guarantee termination, the implementation should employ an anticycling technique, although we have never found this necessary in practice. The resulting spanning tree corresponds to an optimal ranking. For further discussion of the termination of the network simplex algorithm and optimality of the result, the interested reader is referred to the literature [14]–[16].

Fig. 4. describes our version of the network simplex algorithm.

Remarks on Fig. 4

2: The function `feasible_tree` constructs an initial feasible spanning tree. This procedure is described more fully shortly. The simplex method starts with a feasible solution and maintains this invariant.

3: `leave_edge` returns a tree edge with a negative cut value, or nil if there is none, meaning the solution is optimal. Any edge with a negative cut value may be selected as the edge to remove.

4: `enter_edge` finds a nontree edge to replace `e`. This is done by breaking the edge `e`, which divides the tree into a head and tail component. All edges going from the head component to the tail are considered, with an edge of minimum slack being chosen. This is necessary to maintain feasibility.

5: The edges are exchanged, updating the tree and its cut values.

7: The solution is normalized by setting the least rank to zero.

8: Nodes having equal in- and out-edge weights and multiple feasible ranks are moved to a feasible rank with the fewest nodes. The purpose is to reduce crowding and improve the aspect ratio of the drawing, following principle A4. The adjustment does not change the cost of the rank assignment. Nodes are adjusted in a greedy fashion, which works sufficiently well. Globally balancing ranks is considered in a forthcoming paper [14].

```

1. procedure feasible_tree()
2.   init_rank();
3.   while tight_tree() < |V| do
4.     e = a non-tree edge incident on the tree
5.       with a minimal amount of slack;
6.     delta = slack(e);
7.     if incident node is e.head then delta = -delta;
8.     for v in Tree do v.rank = v.rank + delta;
9.   end
10.  init_cutvalues();
11. end

```

Fig. 5. Finding an initial feasible tree.

Fig. 5 shows how to construct an initial feasible tree.

Remarks on Fig. 5

2: An initial feasible ranking is computed. For brevity, `init_rank` is not given here. Our version keeps nodes in a queue. Nodes are placed in the queue when they have no unscanned in-edges. As nodes are taken off the queue, they are assigned the least rank that satisfies their in-edges, and their out-edges are marked as scanned. In the simplest case, where $\delta = 1$ for all edges, this corresponds to viewing the graph as a poset and assigning the minimal elements to rank 0. These nodes are removed from the poset and the new set of minimal elements are assigned rank 1, etc.

3: Function `tight_tree` finds a maximal tree of tight edges containing some fixed node and returns the number of nodes in the tree. Note that such a maximal tree is just a spanning tree for the subgraph induced by all nodes reachable from the fixed node in the underlying undirected graph using only tight edges. In particular, all such trees have the same number of nodes.

4–8: This finds an edge to a nontree node that is adjacent to the tree, and adjusts the ranks of the tree nodes to make this edge tight. As the edge was picked to have minimal slack, the resulting ranking is still feasible. Thus, on every iteration, the maximal tight tree gains at least one node, and the algorithm eventually terminates with a feasible spanning tree. This technique is essentially the one described by Sugiyama *et al* [5].

10: The `init_cutvalues` function computes the cut values of the tree edges. For each tree edge, this is computed by marking the nodes as belonging to the head or tail component, and then performing the sum of the signed weights of all edges whose head and tail are in different components, the sign being negative for those edges going from the head to the tail component.

A small example of running the network simplex algorithm is shown in Fig. 6. Nontree edges are dotted, and all edges have weight 1. In (a), the graph is shown after the initial ranking, with cut values as indicated. For instance, the cut value of edge (g, h) is -1 , corresponding to the weight of edge (g, h) (from the tail component to the head component) minus the weights of edges (a, e) and (a, f) (from the head component

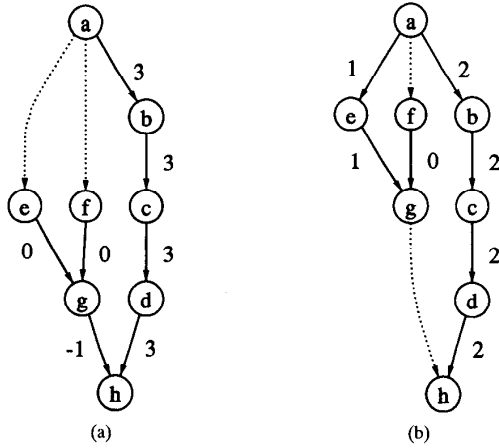


Fig. 6. Finding an optimal feasible tree.

to the tail component). In (b), the edge (g, h) with a negative cut value has been replaced by the nontree edge (a, e) , with the new cut values shown. Because they are all nonnegative, the solution is optimal and the algorithm terminates.

D. Implementation Details

Versions of the network simplex algorithm are well understood and there are results in the literature to help tune an implementation [15]. We feel, however, it is worth pointing out several specific points to prospective implementors. These optimizations are useful here, but become crucial when we use the network simplex again in Section IV, applied to much larger graphs.

Computing the initial feasible tree and initial cut values is frequently a significant proportion of the cost in solving the network simplex algorithm. For many graphs in practice, the initial solution is close to optimal, requiring just a few iterations to reach the final solution. In a naive implementation, initial cut values can be found by taking every tree edge in turn, breaking it, labeling each node according to whether it belongs to the head or tail component, and performing the sum. This takes $O(VE)$ time.

To reduce this cost, we note that the cut values can be computed using information local to an edge if the search is ordered from the leaves of the feasible tree inward. It is trivial to compute the cut value of a tree edge with one of its endpoints a leaf in the tree, since either the head or the tail component consists of a single node. Now, assuming the cut values are known for all the edges incident on a given node except one, the cut value of the remaining edge is the sum of the known cut values plus a term dependent only on the edges incident to the given node.

We illustrate this computation in Fig. 7 in the case where two tree edges, with known cut values, join a third, with the shown orientations. The other cases are handled similarly. We assume the cut values of (u, w) and (v, w) are known. The edges labeled with capital letters represent the set of all nontree edges with the given direction and whose heads and tails belong to the components shown. The cut values of (u, w)

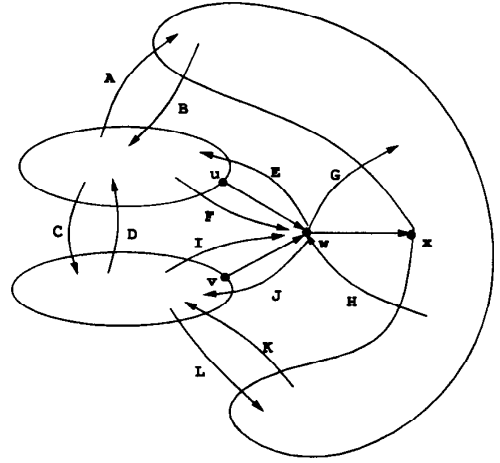


Fig. 7. Incrementally computing cut values.

and (v, w) are given by

$$c_{(u,w)} = \omega(u, w) + A + C + F - B - E - D$$

and

$$c_{(v,w)} = \omega(v, w) + L + I + D - K - J - C$$

respectively. The cut value of (w, x) is then

$$\begin{aligned} c_{(w,x)} &= \omega(w, x) + G - H + A - B + L - K \\ &= \omega(w, x) + G - H + (c_{(u,w)} - \omega(u, w) \\ &\quad - C - F + E + D) \\ &\quad + (c_{(v,w)} - \omega(v, w) - I - D + J + C) \\ &= + \omega(w, x) + G - H + c_{(u,w)} - \omega(u, w) + c_{(v,w)} \\ &\quad - \omega(v, w) - F + E - I + J \end{aligned}$$

an expression involving only local edge information and the known cut values. By thus computing cut values incrementally, we can ensure that every edge is examined only twice. This greatly reduces the time spent computing initial cut values.

Another valuable optimization, similar to a technique described in [15], is to perform a postorder traversal of the tree, starting from some fixed root node v_{root} , and labeling each node v with its postorder traversal number $\text{lim}(v)$, the least number $\text{low}(v)$ of any descendant in the search, and the edge $\text{parent}(v)$ by which the node was reached (see Fig. 8). This provides an inexpensive way to test whether a node lies in the head or tail component of a tree edge, and thus whether a nontree edge crosses between the two components. For example, if $e = (u, v)$ is a tree edge and v_{root} is in the head component of the edge (i.e., $\text{lim}(u) < \text{lim}(v)$), then a node w is in the tail component of e if and only if $\text{low}(u) \leq \text{lim}(w) \leq \text{lim}(u)$. These numbers can also be used to update the tree efficiently during the network simplex iterations. If $f = (w, x)$ is the entering edge, the only edges whose cut values must be adjusted are those in the path connecting w and x in the tree. This path is determined by following the parent edges back from w and x until the least common ancestor is reached, i.e., the first node l such

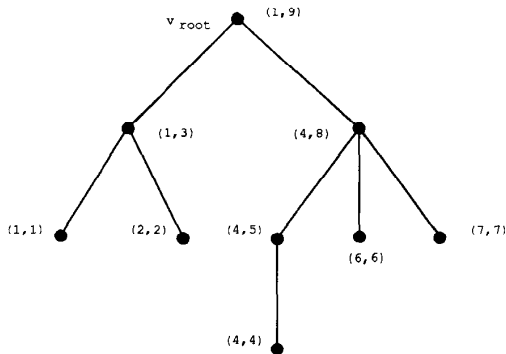


Fig. 8. Postorder traversal with nodes labeled by (low, lim).

that $low(l) \leq lim(w), lim(x) \leq lim(l)$. Of course, these postorder parameters must also be adjusted when exchanging tree edges, but only for nodes below l .

The network simplex is also very sensitive to the choice of the negative edge to replace. We observed that searching cyclically through all the tree edges, instead of searching from the beginning of the list of tree edges every time, can save many iterations.

III. VERTEX ORDERING WITHIN RANKS

After rank assignment, edges between nodes more than one rank apart are replaced by chains of unit length edges between temporary or "virtual" nodes. The virtual nodes are placed on the intermediate ranks, converting the original graph into one whose edges connect only nodes on adjacent ranks. Self-edges are ignored in this pass, and multiedges are merged as in the previous pass.

The vertex order within ranks determines the edge crossings in the layout, thus a good ordering is one with few crossings. Heuristics are appropriate since minimizing edge crossings in layouts of ranked graphs is NP-complete, even for only two ranks [10].

Several important heuristics for reducing edge crossings in ranked graphs are based on the following scheme first suggested by Warfield [3]. An initial ordering within each rank is computed. Then a sequence of iterations is performed to try to improve the orderings. Each iteration traverses from the first rank to the last one, or vice versa. When visiting a rank, each of its vertices is assigned a weight based on the relative positions of its incident vertices on the preceding rank. Then the vertices in the rank are re-ordered by sorting on these weights.

Two common vertex weighting methods are the barycenter function [5] and the median function [17]. Let v be a vertex and P the list of positions of its incident vertices on the appropriate adjacent rank. Note that the position of an adjacent node is only its ordinal number in the current ordering. The barycenter method defines the weight of v as the average of elements in P . The median method defines the weight of v as the median of elements in P . When the number of elements in P is even, there are two medians. This gives rise to two median methods: always using the left median, and always using

```

1. procedure ordering()
2.   order = init_order();
3.   best = order;
4.   for i = 0 to Max_iterations do
5.     wmedian(order,i);
6.     transpose(order);
7.     if crossing(order) < crossing(best) then
8.       best = order;
9.   end
10.  return best;
11. end

```

Fig. 9. Vertex-ordering algorithm.

the right median. The median method consistently performs better than the barycenter method and has a slight theoretical advantage since Eades and Wormald [17] have shown that the median layout of a two-level graph has no more than three times the minimum number of crossings. No such bound is known for the barycenter method.

Our node-ordering heuristic is a refinement of the median method with two major innovations. First, when there are two median values, we use an interpolated value biased toward the side where vertices are more closely packed. The second improvement uses an additional heuristic to reduce obvious crossings after the vertices have been sorted, transforming a given ordering to one that is locally optimal with respect to transposition of adjacent vertices. It typically provides an additional 20–50% reduction in edge crossings. We refer the reader to [1] for detailed statistics.

Fig. 9 shows the node-ordering algorithm.

Remarks on Fig. 9

2: `init_order` initially orders the nodes in each rank. This may be done by a depth-first or breadth-first search starting with vertices of minimum rank. Vertices are assigned positions in their ranks in left-to-right order as the search progresses. This strategy ensures that the initial ordering of a tree has no crossing. This is important because such crossings are obvious, easily avoided "mistakes."

4–9: `Max_iterations` is the maximum number of iterations. We set `Max_iterations` to 24. At each iteration, if the number of crossings improves, the new ordering is saved. In an actual implementation, one might prefer an adaptive strategy that iterates as long as the solution has improved at least a few percent over the last several iterations. `wmedian` reorders the nodes within each rank based on the weighted median heuristic. `transpose` repeatedly exchanges adjacent vertices on the same rank if this decreases the number of crossings. Both of these functions are described more completely as follows.

The weighted median heuristic is shown in Fig. 10. Depending on the parity of the current iteration number, the ranks are traversed from top to bottom or from bottom to top. To simplify the presentation, Fig. 10 only shows one direction in detail.

```

1. procedure wmedian(order,iter)
2.   if iter mod 2 == 0 then
3.     for r = 1 to Max_rank do
4.       for v in order[r] do
5.         median[v] = median_value(v,r-1);
6.       sort(order[r],median);
7.     end
8.   else . . .
9.   endif
10. end
11.
12. procedure median_value(v,adj_rank)
13.   P = adj_position(v,adj_rank);
14.   m = |P|/2;
15.   if |P| = 0 then
16.     return -1.0;
17.   elseif |P| mod 2 == 1 then
18.     return P[m];
19.   elseif |P| = 2 then
20.     return (P[0] + P[1])/2;
21.   else
22.     left = P[m-1] - P[0];
23.     right = P[|P|-1] - P[m];
24.     return (P[m-1]*right + P[m]*left)/(left+right);
25.   endif
26. end

```

Fig. 10. The weighted median heuristic.

Remarks on Fig. 10

1–10: In the forward traversal of the ranks, the main loop starts at rank 1 and ends at the maximum rank. At each rank a vertex is assigned a median based on the adjacent vertices on the previous rank. Then, the vertices in the rank are sorted by their medians. An important consideration is what to do with vertices that have no adjacent vertices on the previous rank. In our implementation such vertices are left fixed in their current positions with nonfixed vertices sorted into the remaining positions.

12–26: The median value of a vertex is defined as the median position of the adjacent vertices if that is uniquely defined. Otherwise, it is interpolated between the two median positions using a measure of tightness. Generally, the weighted median is biased toward the side where vertices are more closely packed.

13: The `adj_position` function returns an ordered array of the present positions of the nodes adjacent to *v* in the given adjacent rank.

```

1. procedure transpose(rank)
2.   improved = True;
3.   while improved do
4.     improved = False;
5.     for r = 0 to Max_rank do
6.       for i = 0 to |rank[r]|-2 do
7.         v = rank[r][i];
8.         w = rank[r][i+1];
9.         if crossing(v,w) > crossing(w,v) then
10.           improved = True;
11.           exchange(rank[r][i],rank[r][i+1]);
12.         endif
13.       end
14.     end
15.   end
16. end

```

Fig. 11. The transposition heuristic for reducing edge crossings.

15–16: Nodes with no adjacent vertices are given a median value of -1 . This is used within the `sort` function to indicate that these nodes should be left in their current positions.

Fig. 11 shows the transposition heuristic.

Remarks on Fig. 11

3–15: This is the main loop that iterates as long as the number of edge crossings can be reduced by transpositions. As in the loop in the `ordering` function, an adaptive strategy could be applied here to terminate the loop once the improvement is a sufficiently small fraction of the number of crossings.

7–12: Each adjacent pair of vertices is examined. Their order is switched if this reduces the number of crossings. The function `crossing(v,w)` simply counts the number of edge crossings if *v* appears to the left of *w* in their rank.

One small point is that the original graph may have edges between nodes on the same rank. We call these “flat edges.” Following criterion A1, we try to aim them all in the same direction across the rank. If ranks are ordered from top to bottom, flat edges generally point from left to right. This involves some minor modifications to the vertex ordering algorithms. If there are flat edges, their transitive closure is computed before finding the vertex order. The vertex order must always embed this partial order. In particular, the initial order must be consistent with it, and the `transpose` and the `sort` routines must not exchange nodes against the partial order.

When sorting nodes by medians and transposing adjacent nodes, equality can occur when comparing median values or number of edge crossings. We have found it helpful, and in keeping with the spirit of A4, to flip nodes with equal values during the sorting or transposing passes on every other forward and backward traversal.

One final point is that it is generally worth the extra cost to run the vertex ordering algorithm twice: once for an initial order determined by starting with vertices of minimal rank and searching out-edges, and the second time by starting with vertices of maximal rank and searching in-edges. This allows one to pick the better of two different solutions.

IV. NODE COORDINATES

The third pass sets node coordinates. Previous work has treated this as a postprocessing step of the barycenter or median methods, making local adjustments to avoid bad layouts. Considering node placement as a separate, well-defined problem, however, yields better layouts and provides a foundation for further extensions, such as trying to set the vertex order by methods that are more topological than geometric.

X and Y coordinates are computed in two separate steps. The first step assigns X coordinates to all nodes (including virtual nodes), subject to the order within ranks already determined. The second step assigns Y coordinates, giving the same value to nodes in the same rank. The Y coordinate assignment maintains the minimum separation $ranksep(G)$ between node boxes. Optionally, the separation between adjacent ranks can be increased to improve the slope of nearly horizontal edges to make them more readable. Because the Y coordinate step is straightforward, the remainder of this section deals with X coordinates.

According to the aesthetic principles already mentioned, short, straight edges are preferable to long, crooked ones. This property of X coordinates is captured in the following integer optimization problem:

$$\begin{aligned} \min \sum_{e=(v,w)} \Omega(e)\omega(e)|x_w - x_v| \\ \text{subject to: } x_b - x_a \geq \rho(a,b) \end{aligned}$$

where a is the left neighbor of b on the same rank and

$$\rho(a,b) = \frac{xsize(a) + xsize(b)}{2} + nodesep(G)$$

The weight $\Omega(e)$, an internal value distinct from the input edge weight $\omega(e)$, is defined to favor straightening long edges. Since edges between real nodes in adjacent ranks can always be drawn as straight lines, it is more important to reduce the horizontal distance between virtual nodes, so chains may be aligned vertically and thus straightened. The failure to straighten long edges can result in a "spaghetti effect" of edges having many different slopes. Accordingly, edges are divided into three types depending on their end vertices 1) both real nodes, 2) one real node and one virtual node, or 3) both virtual nodes. If e , f , and g are edges of types 1)–3), respectively, then $\Omega(e) \leq \Omega(f) \leq \Omega(g)$. Our implementation uses 1, 2, and 8. ρ is a function on pairs of adjacent nodes in the same rank giving the minimum separation between their center points.

There are standard techniques for transforming this problem into a linear program by the addition of auxiliary variables and inequalities to remove the absolute values [15]. As the

```

1. procedure xcoordinate()
2.   xcoord = init_xcoord();
3.   xbest = xcoord;
4.   for i = 0 to Max_iterations do
5.     medianpos(i,xcoord);
6.     minedge(i,xcoord);
7.     minnode(i,xcoord);
8.     minpath(i,xcoord);
9.     packcut(i,xcoord);
10.    if xlength(xcoord) < xlength(xbest) then
11.      xbest = xcoord;
12.    end
13.  return xbest;
14. end

```

Fig. 12. Assigning x coordinates to vertices.

resulting constraints are totally unimodular, solving the linear program with the simplex method produces a solution to the integer program. This is easy to program, and the layouts it gives are aesthetically pleasing. Unfortunately, the transformation increases the size of the simplex matrix from VE to $VE + E^2$ entries. Graphs of a few dozen nodes and edges can be drawn in a few seconds, but larger graphs take much longer, and even the amount of memory available becomes a limitation. So this is not a completely satisfactory way to make layouts, particularly on smaller computers.

A. Heuristic Approach

This approach replaces the linear program with a heuristic for finding X coordinates. The heuristic finds a "good" initial placement, then iteratively tries to improve it by sweeping up and down the ranks similar to the vertex ordering algorithm described in the previous section. The heuristic is sketched in Fig. 12.

Remarks on Fig. 12

2: An initial set of coordinates is computed as follows. For each rank, the left-most node is assigned coordinate 0. The coordinate of the next node is then assigned a value sufficient to satisfy the minimal separation from the previous one, and so on. Thus, on each rank, nodes are initially packed as far left as possible.

4–12: In each iteration, a collection of heuristics is applied to improve the coordinate assignment. If this results in an improvement over the previous best assignment, the coordinates are saved. The function `xlength` implements the objective function from the above optimization problem. In our implementation, `Max_iterations` is 8.

5: The median heuristic is based on the observation that the value $|x - x_0| + |x - x_1| + \dots + |x - x_i|$ is minimized when x is the median of the x_i . The heuristic assigns each node both an upward and downward priority given by the weighted sum of its in- and out-edges, respectively. On downward iterations, nodes are processed in the downward priority order and placed

at the median position of their downward neighbors subject to the placement of higher priority nodes and space requirements of nodes not yet placed. When there are two medians, taking their mean improves symmetry (A4). Upward placement is handled similarly.

6: *minedge* is similar to *medianpos* but considers only edges between two real nodes. It places the edge, oriented vertically, as close as possible to the median of the nodes adjacent to either endpoint of the edge.

7: *minnode* performs local optimization one node at a time, using a queue. Initially all nodes are queued. When a node is removed from the queue, it is placed as close as possible to the median of all its neighbors (both up and down) subject to the separation function ρ . If the node's placement is changed, its neighbors are requeued if not already in the queue. *minnode* terminates when it achieves a local minimum.

8: *minpath* straightens chains of virtual nodes by sequentially finding subchains that may be assigned the same X coordinate.

9: *packcut* sweeps the layout from left to right, searching for blocks that can be compacted. For each node, if all the nodes to the right of it can be shifted to the left by some increment without violating any positioning constraints, the shift is performed. This is performed by an algorithm that operates on a list of nodes sorted in order of X coordinates. Although the algorithm is quadratic in the worst case, it performs well in practice since at every possible cut it only needs to search the nodes in the neighborhood that is affected by the candidate shift.

These heuristics make good layouts quickly, but they are complicated to program and the results are sometimes noticeably imperfect. Further fine tuning is difficult because the heuristics begin to interfere with each other.

B. Optimal Node Placement

We noticed that the *packcut* heuristic does not find all subgraphs that could be compacted to improve the solution. We considered a more general heuristic to search for subgraphs and shift them. We then observed that this is very similar to the way the network simplex algorithm moves entire subgraphs to find an optimal rank assignment (see Section II). This suggested that we apply the network simplex algorithm to find optimal node coordinates, using the X coordinates as "ranks."

The method involves constructing an auxiliary graph as illustrated in Fig. 13. This transformation is the graphical analogue of the algebraic transformation mentioned above for removing the absolute values from the optimization problem. The nodes of the auxiliary graph G' are the nodes of the original graph G plus, for every edge e in G , there is a new node n_e . There are two kinds of edges in G' . One edge class encodes the cost of the original edges. Every edge $e = (u, v)$ in G is replaced by two edges (n_e, u) and (n_e, v) with $\delta = 0$ and $\omega = \omega(e)\Omega(e)$. The other class of edges separates nodes in the same rank. If v is the left neighbor of w , then G' has an edge $f = e_{(v,w)}$ with $\delta(f) = \rho(v, w)$ and $\omega(f) = 0$. This edge forces the nodes to be sufficiently separated but does not affect the cost of the layout.

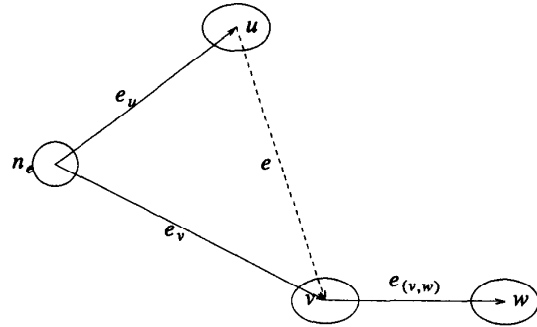


Fig. 13. An auxiliary graph.

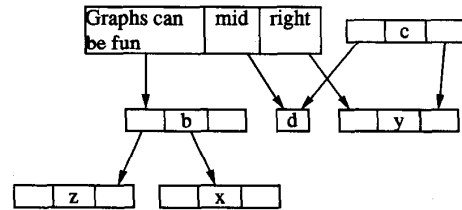


Fig. 14. Node ports in a graph.

We can now consider the level assignment problem on G' , which can be solved using the network simplex method. Any solution of the positioning problem on G corresponds to a solution of the level assignment problem on G' with the same cost. This is achieved by assigning each n_e the value $\min(x_u, x_v)$, using the notation of Fig. 13 and where x_u and x_v are the X coordinates assigned to u and v in G . Conversely, any level assignment in G' induces a valid positioning in G . In addition, in an optimal level assignment, one of e_u or e_v must have length 0, and the other has length $|x_u - x_v|$. This means the cost of an original edge (u, v) in G equals the sum of the cost of the two edges e_u, e_v in G' and, globally, the two solutions have the same cost. Thus, optimality of G' implies optimality for G and solving G' gives us a solution for G .

Using the auxiliary graph also permits the specification of "node ports," or edge endpoints offset in the X direction from the center of the node. This makes it possible to draw pictures of flat records as shown in Fig. 14. When computing coordinates for nodes in these diagrams, the edge lengths must include the displacements of the node ports as well as the distance between the node center points. Let $e = (u, v)$ be an edge and let Δu and Δv be the specified X displacements of the endpoints from the centers of u and v , respectively. A $\Delta < 0$ indicates the port is to the left of the vertex's center. Without loss of generality, assume $\Delta u \leq \Delta v$ and let $d_e = \Delta v - \Delta u$. d_e is a constant since it depends only on the node ports and not the assignments of u and v . We can now solve the same optimization problem, but the cost of edge e is now given by $\Omega(e)\omega(e)|x_v - x_u + d_e|$. In the auxiliary graph, we now set $\delta(e_u) = d_e$ and $\delta(e_v) = 0$. We can then extend the foregoing argument to show that any positioning for G corresponds to a level assignment for G' ; that any optimal level assignment for G' induces a valid positioning for G ;

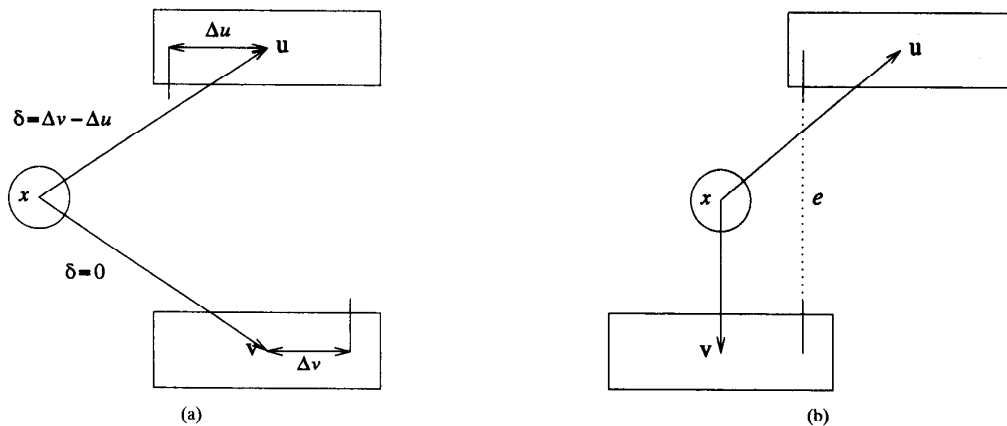


Fig. 15. How port offsets are translated into δ values.

and, in both cases, we have

$$l(e_u) + l(e_v) = |x_v - x_u + d_e| + d_e$$

for all edges (u, v) in G , where l represents the length function in the level assignment on G' . This equation implies that the optimal costs of the problems on G and G' always differ by the constant $\sum_{e \in E} d_e$. Therefore, a minimal assignment for G' corresponds to a minimal assignment for G .

Fig. 15(a) exemplifies how port offsets are translated into the δ value of edges in the auxiliary graph; Fig. 15(b) shows how a solution relates to the original edge.

C. Implementation Details Revisited

The auxiliary graph is considerably larger than the original one. If the original graph has V nodes, E edges, and R ranks, the graph with “virtual” nodes added has $V + D$ nodes and $E + D$ edges, where D is the number of “virtual nodes.” The auxiliary graph then has $V + E + 2D$ nodes and $V + 2E + 3D - R$ edges. This graph requires disproportionately more time to use the network simplex approach. Consequently, the optimizations to the network simplex algorithm described at the end of Section II are crucial for this pass.

Further improvement is possible by noting that it is easy to construct an initial feasible tree for the auxiliary graph by taking advantage of its structure. To construct a feasible tree, use all edges connecting nodes in the same rank. For each pair of adjacent ranks, pick an edge $f = (u, v)$ between the ranks and add both f_u and f_v in G' to the tree. This determines the relative placement of all the nodes in the two ranks. Finally, for every edge $e = (w, x) \neq f$ between the two ranks, add either e_w or e_x to the tree depending on whether w or x is placed leftmost.

Without these improvements, using network simplex to position the nodes took 5–10 times longer. With these improvements, our implementation runs as fast or faster than the heuristic implementation. We do not doubt that the heuristic in turn could also be tuned further, but the real advantage is that the network simplex is much simpler code and produces optimal solutions. Also, improvements that could be difficult to

program into the heuristic can be handled in network simplex. As one example, local symmetry (A4) may be improved by scanning the graph after network simplex terminates. Tree edges whose cut value is exactly 0 identify subgraphs that may be adjusted to equalize the slack on their incident edges without changing the cost of the solution. This could be used to increase symmetry, such as centering a node with an even number of descendants.

V. DRAWING EDGES

In our method, edges are drawn as spline curves. Other graph drawing programs of which we are aware use line segments, and most make no attempt to avoid situations where line segments overlap unrelated nodes. Although splines are more difficult to program, they yield better drawings and help to satisfy aesthetic criterion A2.

In *dag*, edge splines are made by a collection of heuristics that replace the path of line segments between virtual nodes with various straight and curved segments, as described in [1]. The drawback is that the splines sometimes bend sharply to turn inside virtual node boxes or to avoid nearby nodes. The virtual nodes end up being visible in the final layout. This method does not use the available space effectively.

It is better to try to find the smoothest curve between two points that avoids the “obstacles” of other nodes or splines. We can then divide the spline routing algorithm into a top half and a bottom half. The top half computes a polygonal region of the layout where the spline may be drawn. It calls the bottom half to compute the best spline within the region. As a final step, the top half resizes virtual nodes according to the bounding box of the spline, and clips the spline to the boundaries of the endpoint node shapes.

A region and its spline are illustrated in Fig. 16.¹ The associated edge is from “Interdata” to “Unix TS 3.0.” More formally, we draw splines by creating and solving instances of the following subproblem. Given $B_0, \dots, B_m, q, \theta_q, r, \theta_r$ where B_i are boxes parallel to the coordinate axes, such that

¹Graph data courtesy of Ian F. Darwin, SoftQuad Inc., and Geoffrey Collyer, Software Tool & Die.

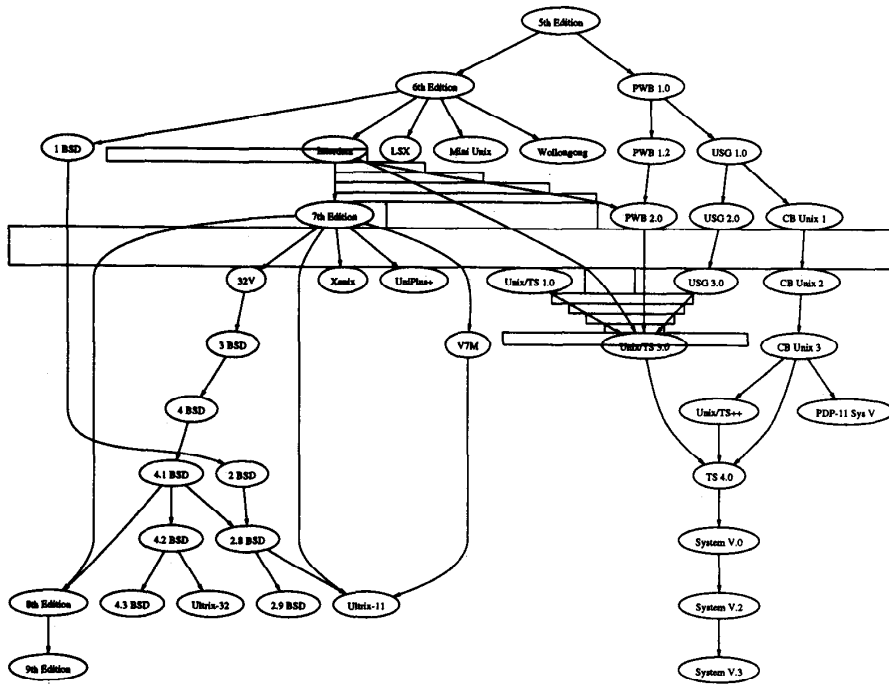


Fig. 16. Region for a spline (0.48 s user time Sun 4/280).

B_i has edges in common with B_{i-1} and B_{i+1} ; q and r are points on or inside the first and last box respectively, find $s_0, \dots, s_n$ and $BB_0, \dots, BB_m$, where s_i are the control points of a piecewise Bezier curve and BB_i are boxes parallel to the coordinate axes. The curve must have q and r as its endpoints. θ_q and θ_r are optional; if they are specified, then the curve must have the given slope at the corresponding endpoint. The BB_i correspond to the B_i and are the smallest boxes that contain the generated splines.

We next describe the two parts of the algorithm.

A. Finding the Region

There are three kinds of edges in the drawing: edges between nodes on different ranks, flat edges between different nodes on the same rank, and self-edges or loops.

Different Ranks: In practice, most edges connect nodes on different ranks. The region for this kind of edge has a few boxes near its tail port, then an alternating sequence of inter-rank boxes and virtual node boxes, and finally a few boxes near the head port. The tail and head port boxes route the spline to the appropriate side of the node.

To curve as smoothly as possible, a spline should be allowed all the space that is available. So the region should include not only virtual node boxes, but also any extra space next to them. After the spline has been computed, the virtual node boxes are updated according to the BB_i , so splines computed afterward will be able to use all the space remaining but not come too close to splines already drawn. Because splines are drawn by a "greedy" strategy, they depend on the order in which they are computed. It seems reasonable to route the shorter splines

first because they can often be drawn as straight lines, but the order does not seem to affect the drawing quality much.

There are three details that can help to improve the appearance of the splines. First, when edges cross, they should not constrain each other too much, otherwise, a spline may have an awkward, sharp turn. This is easily avoided by making an adjustment to the boxes. When setting the size of a box, we ignore virtual nodes to the left or right that correspond to edges that cross within two ranks. Crossings further away are not considered because unintended multiple crossings can occur when the boxes become too sloppy.

Second, when an edge has a section that is almost vertical, it looks better to just draw it as a vertical line. This is most obvious when edges run alongside each other, because parallel line segments look better than long segments with slightly different slopes. When the region finding procedure detects a long vertical section, it terminates the current region, draws its spline, draws the vertical line segment, and finally begins the region of the rest of the edge. This is one of the situations where θ_q and θ_r are used, since the splines must have a vertical tangent at the endpoint where they join the vertical line segment.

Third, when several splines approach a common termination point, it is important to avoid "accidental" intersections. To do this, we check if there are previously computed splines with the same endpoint. If so, we find the closest ones to the right and the left. We then subdivide the inter-rank space, and evaluate the left and right splines at the intervals. These points (or the boundaries of the layout, if one of the left or right splines does not exist) determine a set of boxes that

```

1. procedure compute_splines (B_array, q, theta_q, use_theta_q, s, theta_s, use_theta_s)
2.   compute_L_array (B_array);
3.   compute_p_array (B_array, L_array, q, s);
4.   if use_theta_q then vector_q = anglevector(theta_q)
5.   else vector_q = zero_vector;
6.   if use_theta_s then vector_s = anglevector(theta_s)
7.   else vector_s = zero_vector;
8.   compute_s_array (B_array, L_array, p_array, vector_q, vector_s);
9.   compute_bboxes ();
10. end

```

Fig. 17. Computing splines.

separate the new spline from the existing ones as they approach the terminal node. The left and right splines and the boxes that result can be seen in Fig. 16. This subdivision of the inter-rank box could be viewed as approximating a polygonal region not necessarily aligned with the coordinate axes. In some layouts there are other places where nonaligned boxes or other polygons could prevent unintended tangencies. If we were writing this program again, we would try general polygons instead of boxes.

Thus far we have not mentioned multiple edges between the same pair of nodes. When these exist, a spline is computed for one of the edges, and the rest of the edges are drawn by adding an increasing X coordinate displacement to each one (multiples of $nodesep(G)$ work well). Space for multiple edges must be reserved in the previous pass, described in Section IV, when setting the separation between nodes.

Flat Edges: Flat edges are handled much like inter-rank edges, but the region routes past intervening nodes and spaces between nodes. We omit most of the details since they are quite similar. One difference is that if an edge connects two adjacent nodes it is drawn as a single spline with the following control points:

$$\begin{aligned}
 dx &= (x(u) - x(v)) \\
 p0 &= (x(v), y(v)) \\
 p1 &= p0 + (\tfrac{1}{3}dx, 0) \\
 p2 &= p0 + (\tfrac{2}{3}dx, 0) \\
 p3 &= (x(u), y(u)).
 \end{aligned}$$

For multiple flat edges, a spline is computed for the first one, and succeeding edges are drawn by adding Y coordinate displacements. If an edge has a label, the label is positioned halfway along the edge.

Self-Edges: Self-edges are drawn as loops on the sides of nodes. If an edge specifies tail or head ports, a polygonal region is generated that connects the two ports. The orientation of the region may be either clockwise or counterclockwise, depending on the positions of the ports. If an edge does not specify tail and head ports it is drawn as a sequence of two splines, $p0, \dots, p3$ and $p3, \dots, p6$. These control points are

computed as follows:

$$\begin{aligned}
 dx &= nodesep(G) \\
 dy &= \tfrac{1}{2}ysize(v) \\
 p0 &= (x(v), y(v)) \\
 p1 &= p0 + (\tfrac{1}{3}dx, dy) \\
 p2 &= p0 + (\tfrac{2}{3}dx, dy) \\
 p3 &= p0 + (dx, 0) \\
 p4 &= p0 + (\tfrac{2}{3}dx, -dy) \\
 p5 &= p0 + (\tfrac{1}{3}dx, -dy) \\
 p6 &= p0
 \end{aligned}$$

If there are multiple edges, their loops are nested. If an edge has a label, the label is positioned halfway along the edge. In the simple case mentioned above, the label is positioned to the right of point $p3$. In the case of multiple edges with labels, the sizes of the labels are added to the displacement between edges. This prevents the curve of one edge crossing over the label of another edge. Space for self-edges is allocated in the previous pass, described in Section IV, when setting the separation between adjacent nodes.

B. Computing Splines

The computation of the splines has three stages. First, a piecewise linear curve or path lying entirely inside the region is computed. Then, the endpoints of this path are used as hints in the computation of a piecewise Bezier spline. Finally, the actual space used by the curve is computed in terms of the original boxes. The data structures computed by these three stages are shown in Fig. 19. The region shown in this figure is the same one as in Fig. 16. This example contains 13 boxes.

The three stages are outlined in Fig. 17.

Remarks on Fig. 17

2: **compute_L_array** computes the array $L_0, \dots, L_{m+1}$ where L_1 is the line segment that is the intersection of box B_{i-1} with box B_i . In Fig. 19, these line segments are shown as thicker lines between boxes. There are 14 such segments.

3: **compute_p_array** computes an array of points $p_0, \dots, p_k$ defining a feasible path that connects q and s . In Fig. 19, there are three such points.

```

1. procedure compute_p_array (B_array, L_array, q, s)
2.   if line_fits (B_array, L_array, q, s) then return;
3.   p = compute_linesplit (B_array, L_array);
4.   addto_p_array (p);
5.   compute_p_array (B_array1, L_array1, q, p);
6.   compute_p_array (B_array2, L_array2, p, s);
7. end
8.
9. procedure compute_s_array (B_array, L_array, p_array, vector_q, vector_s)
10.  spline = generate_spline (p_array, vector_q, vector_s);
11.  if size (p_array) == 2 then
12.    while spline_fits (spline, B_array, L_array) == False do
13.      straighten_spline (spline);
14.    elseif spline_fits (spline, B_array, L_array) == False then
15.      count = 0;
16.      ospline = spline;
17.      repeat
18.        spline = refine_spline (p_array, ospline,
19.                                mode (count, max_iterations));
20.        fits = spline_fits (spline, B_array, L_array);
21.        count = count + 1;
22.      while (fits == False) and (count <= max_iterations);
23.      if fits == False then
24.        p = compute_splinesplit (spline, p_array);
25.        compute_s_array (B_array1, L_array1, p_array1,
26.                          vector_q, vector_p);
27.        compute_s_array (B_array2, L_array2, p_array2,
28.                          reverse (vector_p), vector_s);
29.      return;
30.    endif
31.  endif
32.  addto_s_array (spline);
33. end

```

Fig. 18. Spline drawing.

4–7: If `use_theta_q` or `use_theta_s` are true, the curve is constrained to approach the corresponding endpoint at the specified angles. `vector_q` and `vector_s` are normalized vectors.

8: `compute_s_array` computes an array of points $s_0, \dots, s_k$ defining a piecewise Bezier spline that connects q and s and lies entirely inside the region. In the worst case, we can have one Bezier spline per box. In most cases, however, our approach generates significantly fewer splines. For example, in Fig. 19, there are only two splines, one between p_0 and P_1 and one between p_1 and p_2 . In more complex paths,

there may even be fewer splines than line segments, since, unlike a line, a spline can curve around obstacles.

9: `compute_bboxes` computes the space actually taken up by the curve. It computes the array $BB_0, \dots, BB_m$, where BB_i is the narrowest subbox of B_i containing the curve.

`compute_p_array` and `compute_s_array` are both implemented as divide-and-conquer methods, as shown in Fig. 18.

Remarks on Fig. 18

2: `line_fits` checks if the line defined by q and s lies entirely inside the feasible region. The line is clipped to each

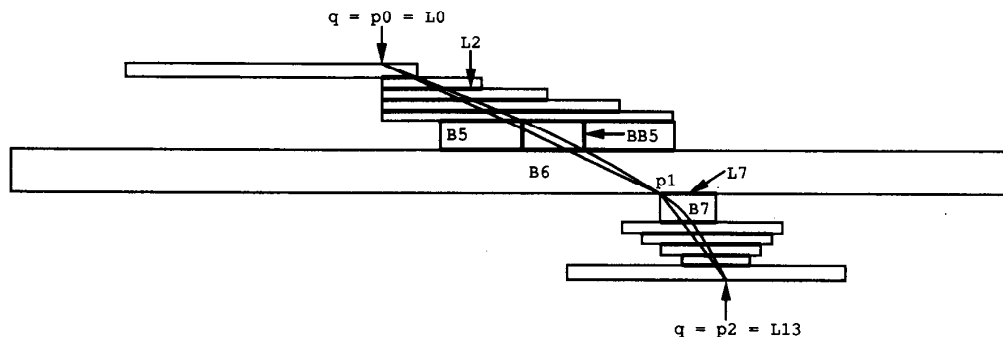


Fig. 19. The three stages.

box; if the line intersects a box, it must do so along the corresponding L segments.

3: If the (q, s) line does not fit, `compute_linesplit` finds the L segment that is the furthest from the (q, s) line and subdivides B_array and L_array along that segment. p is the one of the two endpoints of the subdivision segment that is closer to the (q, s) line. In Fig. 19, for example, the path is subdivided along $L7$.

4: `addto_p_array` adds p to the array of endpoints for the path.

5–6: The two recursive calls to `compute_p_array` complete the computation of the path. `compute_p_array` is not guaranteed to be the shortest path, but it works very well so we have not developed it further. If it were important, the shortest path could be found in linear time using convex hulls [18].

10: `generate_spline` computes a Bezier spline that approximates the path. This is done using a common technique [19].

11–13: The case where there is only one segment in the path is handled first. `spline_fits` checks if the spline lies entirely inside the region. The spline is sampled along its length and these samples are then clipped as a linear path against the box region. The process is similar to that of `line_fits`. As long as the spline does not fit, `straighten_spline` adjusts the control points of the spline to reduce the curvature. In the worst case, the spline becomes a line, and that is known to fit inside the path. This worst case can produce sharp turns. Most of the time, however, the spline fits inside the region after just a few iterations and this process does not produce any visual anomalies. It is only when the region itself makes sharp turns that the worst case may happen.

14–30: The second case is that the path has more than one segment. If the spline does not fit, `refine_spline` perturbs the control points of the spline in an attempt to make the spline fit. The approach is similar to the straightening approach in lines 11–13. We try to decrease the curvature of the spline. If this does not seem to improve the fit, we try to increase the curvature. Since this process may never terminate, `max_iterations` controls how many times to try. `mode` returns a flag to indicate if the curvature is to be increased or decreased. If the spline still does not fit even after the refinement, we subdivide the problem. `compute_splinesplit` finds the endpoint of a segment

on the path that is the furthest from the spline and subdivides the box and path arrays along that point. The two recursive calls to `compute_s_array` compute two piecewise Bezier splines, each fitting inside its corresponding part of the region. To force the two curves to join smoothly at the subdivision point, we also force the two splines to have the same unit tangent vector at that point. This guarantees C^1 continuity at the subdivision point. Forcing C^2 continuity does not seem to produce better results and is also much more expensive to compute. The straightening and refining heuristics, which save a lot of time, are based on the assumption that the tangent vectors at the endpoints of a spline can be scaled independently of the tangent vectors of the two adjacent splines. To maintain C^2 continuity, whenever a tangent vector is scaled, the tangent vector of the adjacent spline must also be scaled so that the two vectors will continue to have the same length. The scaling can propagate all the way to the end of the region. In addition, some of these splines may not fit even after scaling, and this would require more subdivisions, including subdivisions inside a single box. This is more trouble than it is worth.

32: Finally, `addto_s_array` adds the spline to the piecewise Bezier spline.

C. Edge Labels

In *dag*, edge labels are placed next to the midpoint of the spline. This is an oversimplification since the placement does not avoid or even detect overlapping with other splines, labels, or nodes. Yet graphs with edge labels are often small and sparse, so this technique is sometimes adequate.

In *dot*, edge labels on inter-rank edges are represented as off-center virtual nodes. This guarantees that labels never overlap other nodes, edges or labels. Certain adjustments are needed to make sure that adding labels does not affect the length of edges. Setting the minimum edge length to 2 (effectively doubling the ranks when virtual nodes are created) and halving the separation between ranks compensates for the label nodes. This makes it at least twice as expensive to draw a graph with labels, but the labels are readable. Fig. 20 shows a drawing of a graph with edge labels.

Edge labels on self-edges are easy to handle, but flat edges are more complicated. Here we must choose the left-to-right order for the virtual node of the label so that its X coordinate

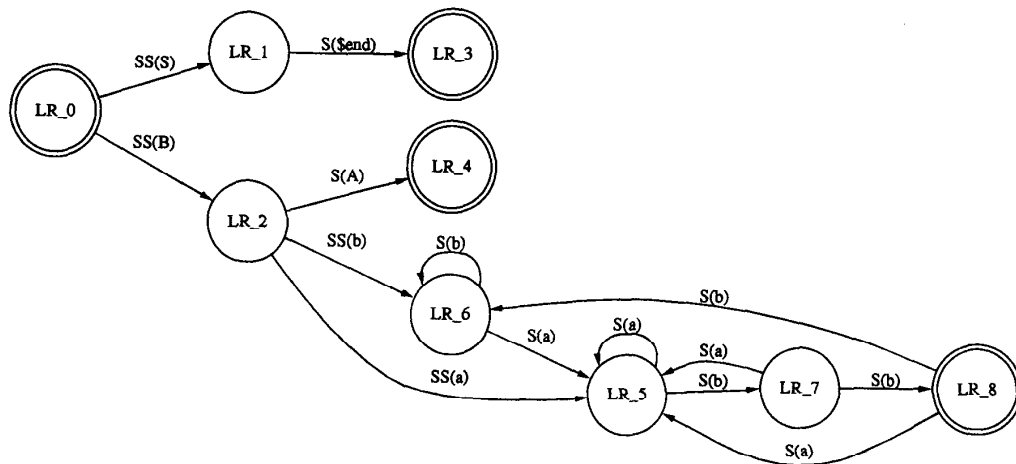


Fig. 20. A finite state machine with labeled transitions (0.15 s user time on a Sun 4/280).

lies between the endpoint coordinates, not to the right or the left. At present we are still working on this problem.

More sophisticated placement of labels in diagrams (such as geographic maps) is a difficult research problem deserving further study. However, it is worth remarking that the label placing program as described by Freeman and Ahn [20] is larger than our whole graph drawing program.

VI. CONCLUSIONS

We have described a method for drawing digraphs. Our contributions are the application of network simplex for assigning ranks and final node coordinates, an improved heuristic for reducing edge crossing, and a method for making edge splines. The method of finding node coordinates allows edges with X coordinate endpoint displacements. These techniques are straightforward to program, run fast enough for interactive use, and make drawings that compare well with previous work as to being readable and visually pleasing.

Further work might address the following:

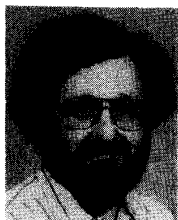
- Understand how to modify the graph or its layout to enhance readability.
- Improve edge-crossing and spline-drawing heuristics.
- Allow more interaction between the layout passes. Different solutions having the same cost in one phase may affect results a great deal in a following phase. For instance, two layouts can have the same number of crossings but much different final coordinates.
- Support incremental (on-line) graph drawing for animation. Stability from one drawing to the next is essential.

ACKNOWLEDGMENT

The referees made detailed comments that helped us to clarify the presentation, particularly in Section II. We also wish to thank G. Jacobson and S. Lally for their criticisms on content and style.

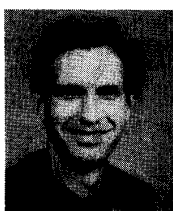
REFERENCES

- [1] E. R. Gansner, S. C. North, and K.-P. Vo, "DAG—A program that draws directed graphs," *Software—Practice and Experience*, vol. 17, no. 1, pp. 1047–1062, 1988.
- [2] P. Eades and R. Tamassia, "Algorithms for automatic graph drawing: an annotated bibliography," *Tech. Rep. CS-89-09* (Revised Version), Brown University, Department of Computer Science, Providence RI, Oct. 1989.
- [3] J. Warfield, "Crossing theory and hierarchy mapping," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-7 no. 7, pp. 505–523, July 1977.
- [4] M. Carpano, "Automatic display of hierarchized graphs for computer aided decision analysis," *IEEE Trans. Software Eng.*, vol. SE-12, no. 4, pp. 538–546, 1980.
- [5] K. S. Sugiyama, S. Tagawa, and M. Toda, "Methods for visual understanding of hierarchical system structures," *IEEE Trans. Syst. Man, Cybern.*, vol. SMC-11, no. 2, pp. 109–125, Feb. 1981.
- [6] G. Di Battista, and R. Tamassia, "Algorithms for plane representations of acyclic digraphs," *Theoretical Computer Sci.* vol. 61, pp. 175–198, 1988.
- [7] E. Koutsofios and S. North, "Drawing graphs with dot," *Tech. Rep.* (available from the authors), AT&T Bell Laboratories, Murray Hill NJ, 1992.
- [8] L. A. Rowe, M. Davis, E. Messinger, C. Meyer, C. Spirakis, and A. Tuan, "A browser for directed graphs," *Software—Practice and Experience*, vol. 17, no. 1, pp. 61–76, Jan. 1987.
- [9] A. Aho, J. Hopcroft, and J. Ullman, *The Design and Analysis of Computer Algorithms*. Reading, MA: Addison-Wesley, 1974.
- [10] P. Eades, B. McKay, and N. Wormald, "On an edge crossing problem," in *Proc. 9th Australian Computer Science Conf.*, 1986, pp. 327–334.
- [11] R. Garey, and D. S. Johnson, *Computers and Intractability*. San Francisco: W. H. Freeman, 1979.
- [12] G. Robbins, "The ISI grapher, a portable tool for displaying graphs pictorially," *Symbolikka '87*, Helsinki, Finland, also *Tech. Rep. IST/RS-87-196*, Information Sciences Institute, Marina Del Rey, CA.
- [13] A. V. Goldberg, and R. E. Tarjan, "Finding minimum-cost circulations by successive approximation," *Mathematics of Operations Res.*, vol. 15, no. 3, pp. 430–466, 1990.
- [14] E. R. Gansner, S. C. North, and K.-P. Vo, "On the rank assignment problem," to be submitted.
- [15] V. Chvatal, *Linear Programming*. New York: W. H. Freeman, 1983.
- [16] W. H. Cunningham, "A network simplex method," *Mathematical Programming*, vol. 11, pp. 105–116, 1976.
- [17] P. Eades and N. Wormald, "The median heuristic for drawing 2-layers networks," *Tech. Rep. 69*, Dept. of Computer Science, University of Queensland, 1986.
- [18] S. Suri, "A linear time algorithm for minimum link paths inside a simple polygon," *Computer Vision, Graphics, and Image Processing*, vol. 35, pp. 99–110, 1986.
- [19] A. S. Glassner Ed., *Graphics Gems*. San Diego: Academic Press, 1990.
- [20] H. Freeman, and J. Ahn, "On the problem of placing names in a geographic map," *Int. J. Pattern Recognition and Artificial Intell.*, vol. 1, no. 1, pp. 121–140, 1987.



Emden R. Gansner received the Ph.D. degree in mathematics from The Massachusetts Institute of Technology, Cambridge, MA, in 1980.

After teaching at the University of Illinois, Urbana, he joined AT&T Bell Labs in 1980 and is currently a Distinguished Member of the Technical Staff. His research interests include graphic user interfaces, programming environments, programming languages, and graphs.



Stephen C. North received the M.A. and Ph.D. degrees in computer science from Princeton University, Princeton, NJ, in 1983 and 1986, respectively.

He has been a Member of the Technical Staff at AT&T Bell Labs since 1980. His current research interests include graph layout programs and algorithms, and interactive programming environments.



Eleftherios Koutsofios received the Diploma in electrical engineering from the National Technical University, Athens, Greece, and the M.A. and Ph.D. degrees in computer science from Princeton University, Princeton, NJ.

He joined AT&T Bell Labs as a Member of the Technical Staff in 1990. His research interests are in the area of interactive techniques, algorithm animation, and computational geometry. He has worked on constraint-based drawing systems, programmable graphics editors, and tools for scientific visualization.



Kiem-Phong Vo received the M.A. and Ph.D. degrees in mathematics from the University of California at San Diego in 1977 and 1981.

He has been with AT&T Bell Laboratories since 1981 and is currently a Distinguished Member of the Technical Staff. His research interests include graph theory, data structures and algorithms, user interface, and software tools. He authored or coauthored several UNIX tools, including the latest System V malloc memory allocation package, `(curses)` library for screen management, and IFS, a language for

building applications with menu and form interfaces.

Dr. Vo was named an AT&T Bell Labs Fellow in 1992.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.